AI/ML Engineer Practical Guide

Oct 15, 2026 · @Manish

1. Python, NumPy, Pandas essentials

Concept: NumPy gives fast vectorised arrays; Pandas gives labelled tables (DataFrames). Almost every practical starts by loading and shaping data with these two.

```
import numpy as np, pandas as pd

a = np.arange(12).reshape(3, 4)   # 3x4 matrix
print(a.mean(axis=0), a @ a.T)   # column means, matrix multiply

df = pd.read_csv("data.csv")
df.info(); df.describe()
df["age"].fillna(df["age"].median(), inplace=True)
df.groupby("city")["salary"].agg(["mean", "count"])
df = pd.get_dummies(df, columns=["city"], drop_first=True)
```

Practical knowledge:

- Avoid Python loops over rows; use vectorised ops or `.apply` only when needed.
- Know `merge`, `groupby`, `pivot_table`, `loc` vs `iloc`, and broadcasting rules.
- Always run `df.shape`, `df.isna().sum()` and `df.dtypes` first.

2. Data preprocessing & EDA

Concept: Models are only as good as their inputs. Handle missing values, encode categories, scale numbers, and split data *before* fitting anything, so no test information leaks into training.

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
import seaborn as sns

sns.heatmap(df.corr(numeric_only=True), annot=True)  # EDA

X, y = df.drop("target", axis=1), df["target"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)

num = X.select_dtypes("number").columns
cat = X.select_dtypes("object").columns
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc",
StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
("oh", OneHotEncoder(handle_unknown="ignore"))]), cat),
])
```

Practical knowledge:

- Fit scalers/imputers on train only; a Pipeline does this for you.
- Use `stratify=y` for imbalanced classes; check class ratio with `y.value_counts(normalize=True)`.
- Outliers: IQR rule or clipping; skewed features: `np.log1p`.
- EDA checklist: histograms, boxplots, correlation heatmap, target distribution.

3. Supervised ML: regression & classification

Concept: Regression predicts a number (price); classification predicts a label (spam / not spam). Start with a simple baseline, then try stronger models.

Algorithm	Type	When to use	Key hyperparameters
Linear / Ridge / Lasso	Regression	Baseline, interpretable	`alpha`

Algorithm	Type	When to use	Key hyperparameters
Logistic Regression	Classification	Baseline, linear boundary	`C`, `penalty`
KNN	Both	Small data, local patterns	`n_neighbors`
Decision Tree	Both	Explainable rules	`max_depth`, `min_samples_leaf`
Random Forest	Both	Strong default on tabular data	`n_estimators`, `max_depth`
XGBoost / LightGBM	Both	Best accuracy on tabular data	`learning_rate`, `n_estimators`, `max_depth`
SVM	Both	Medium data, clear margins	`C`, `kernel`, `gamma`

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

for name, model in {"logreg": LogisticRegression(max_iter=1000),
                    "rf": RandomForestClassifier(n_estimators=300,
random_state=42)}.items():
    clf = Pipeline([("pre", pre), ("model", model)])
    clf.fit(X_tr, y_tr)
    print(name, clf.score(X_te, y_te))
```

Practical knowledge:

- Linear models need scaled features; tree models don't.
- Report a baseline (e.g. `DummyClassifier`) so examiners see your improvement.
- Be able to explain gradient descent, entropy/Gini, and bagging vs boosting.

4. Model evaluation & tuning

Concept: Pick the metric that matches the business cost of errors, validate with cross-validation, then tune hyperparameters.

Task	Metric	Use when
Classification	Accuracy	Classes balanced
Classification	Precision	False positives are costly (spam filter)
Classification	Recall	False negatives are costly (disease, fraud)
Classification	F1, ROC-AUC	Imbalanced data
Regression	MAE	Errors in original units, robust to outliers
Regression	RMSE	Large errors should hurt more
Regression	R^2	Share of variance explained

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix

print(cross_val_score(clf, X_tr, y_tr, cv=5, scoring="f1").mean())

grid = GridSearchCV(clf, {"model__n_estimators": [100, 300],
                          "model__max_depth": [None, 10, 20]},
                    cv=5, scoring="f1", n_jobs=-1)
grid.fit(X_tr, y_tr)
pred = grid.best_estimator_.predict(X_te)
print(grid.best_params_)
print(confusion_matrix(y_te, pred)); print(classification_report(y_te, pred))
```

Practical knowledge:

- Overfitting = high train, low test score: regularise, simplify, or add data.
- Imbalance: `class_weight="balanced"`, SMOTE, or threshold tuning.
- Touch the test set only once, at the very end.

5. Unsupervised learning

Concept: No labels. Clustering groups similar points (customer segments); dimensionality reduction compresses features while keeping most information (PCA).

```
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

Xs = StandardScaler().fit_transform(X_num)
inertia = [KMeans(k, n_init=10, random_state=42).fit(Xs).inertia_ for k in
range(2, 11)]  # elbow
km = KMeans(4, n_init=10, random_state=42).fit(Xs)
print(silhouette_score(Xs, km.labels_))

pca = PCA(n_components=0.95).fit(Xs)   # keep 95% variance
print(pca.n_components_, pca.explained_variance_ratio_)
```

Practical knowledge:

- Always scale before K-Means and PCA.
- Choose k with the elbow plot plus silhouette score (closer to 1 is better).
- DBSCAN handles odd shapes and outliers; Isolation Forest is a go-to for anomaly detection.

6. Deep learning with PyTorch

Concept: A neural network stacks layers of weights + non-linear activations. Training repeats: forward pass → loss → backpropagation (gradients) → optimizer step. Memorise this training loop; it appears in nearly every DL practical.

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"
ds = TensorDataset(torch.tensor(Xtr, dtype=torch.float32), torch.tensor(ytr))
dl = DataLoader(ds, batch_size=64, shuffle=True)

model = nn.Sequential(nn.Linear(Xtr.shape[1], 128), nn.ReLU(),
nn.Dropout(0.3),
                      nn.Linear(128, n_classes)).to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(20):
    model.train()
```

```
    for xb, yb in dl:
        xb, yb = xb.to(device), yb.to(device)
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
    print(epoch, loss.item())

model.eval()
with torch.no_grad():
    preds = model(torch.tensor(Xte,
dtype=torch.float32).to(device)).argmax(1)
```

Practical knowledge:

- `model.train()` / `model.eval()` switch Dropout and BatchNorm behaviour.
- Loss not falling: lower the learning rate, check labels, normalise inputs.
- Use early stopping on validation loss; save with `torch.save(model.state_dict(), "m.pt")`.
- Know activations (ReLU, sigmoid, softmax), optimizers (SGD, Adam), and vanishing gradients.

7. CNNs and NLP/Transformers

Concept (CNN): Convolution filters slide over images to detect edges, then shapes, then objects; pooling shrinks the map. In practice, fine-tune a pretrained model (transfer learning) instead of training from scratch.

```
from torchvision import models, transforms, datasets

tf = transforms.Compose([transforms.Resize((224, 224)),
transforms.ToTensor(),
                         transforms.Normalize([0.485, 0.456, 0.406], [0.229,
0.224, 0.225])])
train = datasets.ImageFolder("data/train", transform=tf)

model = models.resnet18(weights="DEFAULT")
for p in model.parameters(): p.requires_grad = False   # freeze backbone
model.fc = nn.Linear(model.fc.in_features, len(train.classes))
# then reuse the training loop from section 6
```

Concept (NLP): Text becomes tokens, then vectors. Classic: TF-IDF + Logistic Regression. Modern: Transformers use self-attention so each word weighs every other word.

```
from sklearn.feature_extraction.text import TfidfVectorizer
txt_clf = Pipeline([("tfidf", TfidfVectorizer(ngram_range=(1, 2))), ("lr",
LogisticRegression())])

from transformers import pipeline
sentiment = pipeline("sentiment-analysis")
print(sentiment("The model trained perfectly!"))
```

Practical knowledge:

- Use data augmentation (flips, crops) for small image datasets.
- Explain attention, embeddings, tokenisation, and BERT (encoder) vs GPT (decoder).
- Fine-tune with Hugging Face `Trainer` when the task needs custom labels.

8. LLM apps & RAG

Concept: RAG (Retrieval-Augmented Generation) lets an LLM answer from your documents: split docs into chunks, embed them, store vectors, retrieve the closest chunks for a question, then pass them to the LLM as context.

1. Load and chunk documents (≈500 tokens, small overlap)
2. Embed chunks and store in a vector index (FAISS, Chroma)
3. Embed the question, retrieve top-k chunks
4. Prompt the LLM with question + chunks; ask it to cite them

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np, anthropic

emb = SentenceTransformer("all-MiniLM-L6-v2")
vecs = emb.encode(chunks, normalize_embeddings=True)
index = faiss.IndexFlatIP(vecs.shape[1]); index.add(vecs)

def ask(q, k=3):
    _, ids = index.search(emb.encode([q], normalize_embeddings=True), k)
    context = "\n\n".join(chunks[i] for i in ids[0])
    client = anthropic.Anthropic()
    msg = client.messages.create(model="claude-sonnet-5-5", max_tokens=500,
        messages=[{"role": "user", "content": f"Answer only from this
context:\n{context}\n\nQ: {q}"}])
    return msg.content[0].text
```

Practical knowledge:

- Know prompt engineering, temperature, context windows, and hallucination control.
- Evaluate retrieval (does the right chunk appear in top-k?) separately from answer quality.
- Also know fine-tuning vs RAG, and basic tool use / agents.

9. Deployment & MLOps

Concept: A model is useful only when served. Save the trained pipeline, wrap it in an API, containerise it, and track experiments and data drift.

```
# train.py
import joblib; joblib.dump(grid.best_estimator_, "model.joblib")

# app.py  ->  run: uvicorn app:app --reload
from fastapi import FastAPI
import joblib, pandas as pd
app = FastAPI()
model = joblib.load("model.joblib")

@app.post("/predict")
def predict(data: dict):
    return {"prediction": model.predict(pd.DataFrame([data])).tolist()}
```

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Practical knowledge:

- Save the whole Pipeline, not just the model, so preprocessing travels with it.
- Track runs with MLflow; version data with DVC; use Git for code.
- Know Streamlit for quick demos, plus monitoring for data and concept drift.

10. Practical exam tips & viva questions

Exam flow that scores well: load → EDA → preprocess → baseline → 2–3 models → tune → evaluate → conclude in 2 lines.

- Comment your code and print shapes/metrics at each step.
- Set `random_state=42` everywhere for reproducibility.
- Keep a personal snippets file (sections 1–9) and practise typing it from memory.
- Use Claude to generate mock tasks, review your code, and quiz you; solve first, then ask.

Common viva questions:

- [ ] Bias vs variance; how do you detect and fix overfitting?
- [ ] Precision vs recall: which matters for fraud detection and why?
- [ ] How does Random Forest differ from Gradient Boosting?
- [ ] Why scale features? Which models need it?
- [ ] Explain backpropagation and the role of the learning rate.
- [ ] What is data leakage? Give an example.
- [ ] L1 vs L2 regularisation.
- [ ] How does self-attention work? What problem does RAG solve?
- [ ] How would you deploy and monitor a model in production?